

Descargas necesarias

Instalar las herramientas base

```
winget install --id Git.Git -e --source winget
```

```
winget install --id EclipseAdoptium.Temurin.8.JDK -e --source winget
```

```
winget install --id Apache.Maven -e --source winget
```

```
winget install --id Docker.DockerDesktop -e --source winget
```

FASE 1

Requisito

Asegurar que Docker Desktop esté instalado y configurado para usar Contenedores de Linux (WSL 2), no contenedores de Windows." (Esto es vital, porque Postgres y Nginx corren en Linux).

Cómo asegurar el "Modo Linux" en Docker Desktop

Abre el Dashboard de Docker: Haz clic en el ícono de la ballena en la barra de tareas de Windows.

Entra a Settings (Engranaje): Haz clic en el ícono de engranaje arriba a la derecha.

Configuración General:

- En la pestaña General, asegúrate de que la casilla "Use the WSL 2 based engine" esté marcada. (Si no lo está, marcala y dale a *Apply & Restart*).

Integración WSL:

- Ve a Resources > WSL Integration.
- Aquí verás una lista de tus distribuciones de Linux instaladas (por ejemplo, **Ubuntu** o **Debian**). Asegúrate de que el interruptor esté activado para la distribución que estás usando (o activa "Enable integration with my default WSL distro").
- Dale a *Apply & Restart*.

2. Averigua la IP estática que tendrá nuestra aplicación y abre el Firewall (En PowerShell): Conéctate por Escritorio Remoto. Abre el menú Inicio, busca **PowerShell**, dale clic derecho y selecciona **"Ejecutar como Administrador"**.

- **Para ver la IP del servidor:**

```
ipconfig
```

(Anota la "Dirección IPv4" de tu adaptador Ethernet, ej. *10.50.1.200*).

Para abrir las puertas del Firewall de Windows:

```
New-NetFirewallRule -DisplayName "Johnson APQP HTTP/HTTPS" -Direction Inbound  
-LocalPort 80,443 -Protocol TCP -Action Allow
```

FASE 2: Descarga en el Directorio

Todo esto lo sigues ejecutando en tu ventana de PowerShell como Administrador:

Crea el directorio:

```
mkdir C:\Apps\Sistema_Johnson
```

```
cd C:\Apps\Sistema_Johnson
```

1. **Clona el repositorio:**

```
git clone https://github.com/Francosg25/Sistema_Johnson.git .
```

FASE 3: Bóveda de Seguridad (Certificados SSL en Windows)

Crea la carpeta de certificados

```
mkdir certs
```

1. **ESCENARIO A** Copiar los archivos *johnson.crt* y *johnson.key* desde tu correo corporativo o USB, y pegarlos usando el Explorador de Archivos de Windows directamente en *C:\Apps\Sistema_Johnson\certs*.

2. **ESCENARIO B** Si TI no los tiene, usamos Docker. Usaremos unos autogenerados, la conexión dirá insegura, pero es algo temporal:

```
docker run --rm -v "${PWD}/certs:/certs" alpine/openssl req -x509 -nodes -days 3650  
-newkey rsa:2048 -keyout /certs/johnson.key -out /certs/johnson.crt -subj  
"/C=MX/ST=Zacatecas/L=Zacatecas/O=JohnsonElectric/OU=Dev/CN=IP_DEL_SERVIDOR"
```

FASE 4: Inyección de Variables

Abre el archivo en el Bloc de Notas:

notepad .env

Fragmento de código:

DB_NAME=johnsondb

DB_USER=admin

DB_PASSWORD=johnsonbase2026

DEFAULT_ADMIN_PASSWORD=adminpassjohnson

APP_BASE_URL=https://[IP_ESTATICA_DEL_SERVIDOR]

SPRING_MAIL_HOST=smtp.gmail.com

SPRING_MAIL_PORT=587

MAIL_USERNAME=johnsonelectricapqp@gmail.com

MAIL_PASSWORD=jviahxfgodjajgnp

(En el Bloc de Notas, dale a Archivo > Guardar, y luego ciérralo).

FASE 5

Levanta la infraestructura (desde PowerShell):

PowerShell

```
docker compose up -d --build
```

Supervisa el arranque:

PowerShell

```
docker compose logs -f app
```

La Prueba de Fuego: Espera el **Started JohnsonSistemaApplication**. Abre el navegador Edge o Chrome dentro de ese mismo Windows Server y teclea **<https://localhost>** o la IP de la máquina.

ESCENARIO FUTURO CON CLOUDFLARE (CERTIFICADOS SSL Y DOMINIO)

Aquí tienes el tutorial definitivo y estructurado para cuando decidas abrir el sistema de APQP al exterior.

Fase 1: Requisitos Previos

Antes de tocar cualquier configuración, necesitarás tener listos estos elementos:

1. **Dominio Corporativo:** El departamento de IT debe proporcionarte el dominio o subdominio oficial que usarán (por ejemplo, [apqp.empresa.com](#)).
2. **Cuenta de Cloudflare:** El dominio proporcionado por IT debe tener sus DNS gestionados en una cuenta de Cloudflare.
3. **Acceso al Server:** Permisos de administrador en el servidor donde actualmente corre tu stack de Docker.

Fase 2: Configuración en el Panel de Cloudflare

Esta parte se hace enteramente en el navegador web.

1. Inicia sesión en Cloudflare y dirígete a la sección **Zero Trust** (en el menú lateral izquierdo).
2. Navega a **Networks > Tunnels**.
3. Haz clic en el botón **Create a tunnel**.
4. Selecciona el conector **Cloudflared** y haz clic en Next.
5. Asigna un nombre descriptivo al túnel (ej. [tunnel-produccion-apqp](#)) y haz clic en Save.
6. En la pantalla de instalación, elige la pestaña **Docker**.
7. Verás un comando largo. Identifica la cadena de texto que está justo al final, después del parámetro `--token`. Copia ese **Token secreto** y guárdalo.

Fase 3: Integración en tu Servidor Local

Aprovechando que tu infraestructura ya está orquestada y empaquetada, la forma más limpia es inyectar el túnel en tu red actual.

1. Abre tu archivo [docker-compose.yml](#) en el Windows Server.

2. Añade el nuevo servicio al final del archivo, alineado con tus servicios **db**, **app** y **nginx**:

cloudflared:

image: cloudflare/cloudflared:latest

container_name: cloudflared_tunnel

restart: always

environment:

- TUNNEL_TOKEN=AQUI_PEGA_EL_TOKEN_QUE_COPIASTE

command: tunnel run

depends_on:

- nginx

logging:

driver: "json-file"

options:

max-size: "10m"

max-file: "3"

3. Abre una terminal en el servidor y actualiza los contenedores para levantar el túnel: **docker-compose up -d**
4. Regresa al panel de Cloudflare. El estado de tu túnel ahora debería decir **Healthy**.

Fase 4: Enrutamiento del Tráfico (Routing)

Ahora le dirás a Cloudflare hacia dónde enviar a los usuarios.

1. En el panel de Cloudflare, haz clic en **Next** (o ve a la pestaña **Public Hostname** dentro de tu túnel).
2. Haz clic en **Add a public hostname**.
3. Completa los campos:
 - **Subdomain:** Escribe el subdominio si aplica (ej. **apqp**).
 - **Domain:** Selecciona el dominio oficial dado por IT en la lista desplegable.

- **Type:** Selecciona **HTTPS**.
 - **URL:** Escribe **nginx:443**.
4. Expande la sección **Additional application settings** justo debajo.
 5. Ve a la pestaña **TLS** y activa la opción **No TLS Verify** (esto es crucial para que Cloudflare acepte los certificados locales que montaste en Nginx).
 6. Haz clic en **Save hostname**.
-

Fase 5: Ajustes Finales en la Aplicación

Para que las vistas de Thymeleaf y los hipervínculos en los correos de notificación se construyan con la nueva dirección pública y no con la IP interna (**10.240.4.21**), debes actualizar el entorno.

1. Abre tu archivo oculto **.env**.
2. Modifica la variable de la URL base con el dominio exacto que configuraste en Cloudflare: **APP_BASE_URL=https://apqp.empresa.com**
3. Reinicia tu aplicación Spring Boot para que absorba el nuevo valor:
docker-compose down && docker-compose up -d

A partir de este punto, la plataforma estará disponible globalmente bajo el dominio oficial, protegida por los servidores de Cloudflare y sin haber expuesto un solo puerto físico en las instalaciones de la empresa.